

```

import { useState, useRef } from "react";

const DEFAULT_ITEMS = [
  { id: 1, label: "", done: false },
  { id: 2, label: "", done: false },
  { id: 3, label: "", done: false },
  { id: 4, label: "", done: false },
  { id: 5, label: "", done: false },
  { id: 6, label: "", done: false },
];

function HorizontalToggle({ done, onToggle }) {
  return (
    <div
      onClick={onToggle}
      style={{
        width: 96,
        height: 44,
        borderRadius: 22,
        background: "#ddd",
        boxShadow: "inset 0 2px 5px rgba(0,0,0,0.18)",
        position: "relative",
        cursor: "pointer",
        overflow: "hidden",
        userSelect: "none",
        flexShrink: 0,
      }}
    >
      { /* Left half - red X */ }
      <div style={{
        position: "absolute", left: 0, top: 0, bottom: 0, width: "50%",
        background: "#c0392b", borderRadius: "22px 0 0 22px",
        display: "flex", alignItems: "center", justifyContent: "center",
        opacity: done ? 0.2 : 1, transition: "opacity 0.22s",
      }}>
        <span style={{ color: "#fff", fontSize: 18, fontWeight: 900, lineHeight:
      </div>

      { /* Right half - blue O */ }
      <div style={{
        position: "absolute", right: 0, top: 0, bottom: 0, width: "50%",
        background: "#3498db", borderRadius: "0 22px 22px 0",
        display: "flex", alignItems: "center", justifyContent: "center",
        opacity: done ? 1 : 0.2, transition: "opacity 0.22s",

```

```

    >>
    <div style={{ width: 20, height: 20, borderRadius: "50%", border: "3px so
</div>

    { /* Puck */ }
    <div style={{
      position: "absolute", top: "50%", transform: "translateY(-50%)",
      left: done ? 4 : "calc(100% - 44px)",
      transition: "left 0.22s cubic-bezier(.4,1.3,.6,1)",
      width: 40, height: 36, borderRadius: 18,
      background: "linear-gradient(145deg, #ffffff, #e2e2e2)",
      boxShadow: "0 2px 6px rgba(0,0,0,0.25)",
      zIndex: 10,
    }} />
  </div>
);
}

function EditModal({ item, onSave, onDelete, onClose }) {
  const [label, setLabel] = useState(item.label);
  return (
    <div onClick={onClose} style={{
      position: "fixed", inset: 0, background: "rgba(0,0,0,0.45)",
      display: "flex", alignItems: "center", justifyContent: "center", zIndex: 10
    }}>
      <div onClick={e => e.stopPropagation()} style={{
        background: "#fff", borderRadius: 20, padding: "28px 24px 20px",
        minWidth: 260, boxShadow: "0 16px 48px rgba(0,0,0,0.2)",
        display: "flex", flexDirection: "column", gap: 16,
      }}>
        <p style={{ margin: 0, fontWeight: 800, fontSize: 15, color: "#333",
          fontFamily: "'Hiragino Kaku Gothic ProN','Noto Sans JP',sans-serif" }}>
          ラベルを編集
        </p>
        <input
          autoFocus value={label}
          onChange={e => setLabel(e.target.value)}
          placeholder="例：タオル" maxLength={8}
          style={{
            border: "2px solid #ddd", borderRadius: 10,
            padding: "10px 14px", fontSize: 16,
            fontFamily: "'Hiragino Kaku Gothic ProN','Noto Sans JP',sans-serif",
            outline: "none",
          }}
          onFocus={e => (e.target.style.borderColor = "#3498db")}
          onBlur={e => (e.target.style.borderColor = "#ddd")}
        />

```

```

    <div style={{ display: "flex", gap: 8 }}>
      <button onClick={() => onSave(label)} style={{
        flex: 1, padding: "10px 0", borderRadius: 10,
        border: "none", background: "#3498db", color: "#fff",
        fontWeight: 700, fontSize: 14, cursor: "pointer", fontFamily: "inheri
      }}>保存</button>
      <button onClick={onDelete} style={{
        flex: 1, padding: "10px 0", borderRadius: 10,
        border: "none", background: "#f0f0f0", color: "#c0392b",
        fontWeight: 700, fontSize: 14, cursor: "pointer", fontFamily: "inheri
      }}>削除</button>
    </div>
  </div>
</div>
);
}

```

```

export default function App() {
  const [items, setItems] = useState(DEFAULT_ITEMS);
  const [editingId, setEditingId] = useState(null);
  const nextId = useRef(100);

  const toggle = id =>
    setItems(prev => prev.map(it => it.id === id ? { ...it, done: !it.done } : it

  const resetAll = () =>
    setItems(prev => prev.map(it => ({ ...it, done: false })));

  const addItem = () => {
    const id = nextId.current++;
    setItems(prev => [...prev, { id, label: "", done: false }]);
    setEditingId(id);
  };

  const saveLabel = (id, label) => {
    setItems(prev => prev.map(it => it.id === id ? { ...it, label } : it));
    setEditingId(null);
  };

  const deleteItem = id => {
    setItems(prev => prev.filter(it => it.id !== id));
    setEditingId(null);
  };

  const editingItem = items.find(it => it.id === editingId);
  const doneCount = items.filter(it => it.done).length;
  const allDone = items.length > 0 && doneCount === items.length;

```

```

return (
<div style={{
  minHeight: "100vh", background: "#e8e8e8",
  display: "flex", flexDirection: "column",
  alignItems: "center", justifyContent: "center",
  padding: 24,
  fontFamily: "'Hiragino Kaku Gothic ProN','Noto Sans JP',sans-serif",
}}>
  <div style={{
    background: "#f4f4f4", borderRadius: 28,
    padding: "22px 20px 18px", width: "100%", maxWidth: 360,
    boxShadow: "0 8px 32px rgba(0,0,0,0.14), 0 2px 8px rgba(0,0,0,0.07)",
  }}>
    { /* Header */ }
    <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between" }}>
      <div>
        <p style={{ margin: 0, fontSize: 17, fontWeight: 800, color: "#222" }}>
          忘れ物チェック
        </p>
        <p style={{ margin: "2px 0 0", fontSize: 11, color: "#aaa" }}>
          {doneCount} / {items.length} 完了
        </p>
      </div>
      {allDone && (
        <span style={{
          fontSize: 11, background: "#2ecc71", color: "#fff",
          padding: "4px 10px", borderRadius: 20, fontWeight: 700,
        }}>✓ 完了!</span>
      )}
    </div>

    { /* Rows */ }
    <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
      {items.map(item => (
        <div key={item.id} style={{ display: "flex", alignItems: "center", gap: 10 }}>
          <HorizontalToggle done={item.done} onToggle={() => toggle(item.id)} />
          <div>
            onClick={() => setEditingId(item.id)}
            style={{
              flex: 1, height: 44, background: "#fff",
              border: "1.5px solid #ddd", borderRadius: 10,
              display: "flex", alignItems: "center", paddingLeft: 14,
              cursor: "pointer", boxSizing: "border-box",
            }}
            >
            <span style={{

```

```

        fontSize: 14, fontWeight: 700,
        color: item.label ? (item.done ? "#bbb" : "#333") : "#ccc",
        textDecoration: item.done && item.label ? "line-through" : "none",
        textDecorationColor: "#bbb",
      }}>
      {item.label || "タップして入力"}
    </span>
  </div>
</div>
))}
</div>

{/* Buttons */}
<div style={{ display: "flex", gap: 8, marginTop: 18 }}>
  <button onClick={resetAll} style={{
    flex: 1, padding: "12px 0", borderRadius: 12,
    border: "none", background: "#c0392b", color: "fff",
    fontWeight: 700, fontSize: 13, cursor: "pointer",
    letterSpacing: "0.05em", fontFamily: "inherit",
  }}>x リセット</button>
  <button onClick={addItem} style={{
    flex: 1, padding: "12px 0", borderRadius: 12,
    border: "none", background: "#3498db", color: "fff",
    fontWeight: 700, fontSize: 13, cursor: "pointer",
    letterSpacing: "0.05em", fontFamily: "inherit",
  }}>+ 追加</button>
</div>

</div>

(editingItem && (
  <EditModal
    item={editingItem}
    onSave={label => saveLabel(editingItem.id, label)}
    onDelete={() => deleteItem(editingItem.id)}
    onClose={() => setEditingId(null)}
  />
))
</div>
);
}

```